

Evaluación Parcial N° 3

Encargo | Estudiante

Subdirección de Diseño
Curricular e Instruccional
2025

MALETA DIDÁCTICA DuocUC

Evaluación Parcial N° 3

Encargo | Estudiante

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación	Semana inicio EFT
DSY1107	Desarrollo Cloud Native I	5 horas pedagógicas	12%	

1. Instrucciones generales

Descripción

- En esta etapa deberás incorporar RabbitMQ al sistema desarrollado previamente, implementando colas, exchanges y DLQs para gestionar tareas asíncronas como notificaciones, generación de tickets o documentos.
- Tu desafío es demostrar que la mensajería desacoplada funciona correctamente, sin afectar la lógica existente del sistema.
- La distribución de los porcentajes en esta evaluación es la siguiente:

Evaluación	Tipo de situación evaluativa	Distribución de porcentajes	Individual/Grupal
Evaluación Parcial N°3 (12%)	Encargo	100%	Parejas
Total		100%	

- El encargo corresponde al código de los componentes frontend y backend del sistema, entregado vía GitHub.
- El tiempo asignado para desarrollar esta evaluación es de 2 semanas para la elaboración del encargo y de la presentación.
- Las/los estudiantes inician el trabajo del encargo en el Taller de Proyectos (Taite 7), pero deben finalizarlo junto a presentación en su tiempo de trabajo personal.

Instrucciones específicas de la Evaluación:

- Los componentes desarrollados, frontend y backend se entregan mediante el envío a el/la docente del código fuente.
- El código de todos los componentes del backend debe compilar, seguir buenas prácticas y responder a pruebas básicas.
- El código del frontend debe estar completo, modular, sin errores de compilación y con vistas funcionales.
- La integración del backend con la base de datos cloud debe estar configurada correctamente mediante entidades, repositorios y propiedades de conexión.
- El backend debe incluir filtros que validen el JWT recibido desde el IDaaS para autorizar las peticiones.
- El frontend debe implementar el flujo de login con IDaaS y utilizar el JWT en las llamadas al backend.
- Para ambos contextos, front y back, solo debe subirse a los repositorios lo que corresponda a la tecnología utilizada, configurando los archivos .gitignore necesarios para lograr esto.
- Los microservicios deben incluir en su código productores y consumidores que se conecten a colas RabbitMQ.
- Los microservicios deben manejar mensajes no entregados enviándolos a DLQ y registrándolos en logs.
- El microservicio administrador de RabbitMQ debe permitir la creación y eliminación de colas, exchanges y bindings.

Los aspectos formales son:

- Formato: el backend debe corresponder a varios microservicios construidos en Java con Spring Boot, el frontend debe ser un componente Angular. Ambos tipos de componentes deben ser entregados como enlaces a Github.
- Medio de entrega: Copiar los enlaces de los repositorios de Github a AVA y enviar copia al correo del docente, en el plazo de entrega establecido.

Los materiales, herramientas o insumos que se requieren para realizar esta evaluación:

- Materiales y contenidos académicos proporcionados durante el curso, incluyendo especificaciones técnicas, análisis previos y guías de referencia.
- Computadores disponibles en taller de proyectos (TAITE 7)

2. Pauta de Evaluación

Categoría	% logro	Descripción niveles de logro
Muy buen desempeño	**100%**	Demuestra un desempeño destacado, evidenciando el logro de todos los aspectos evaluados en el indicador.
Buen desempeño	**80%**	Demuestra un alto desempeño del indicador, presentando pequeñas omisiones, dificultades y/o errores.
Desempeño aceptable	**60%**	Demuestra un desempeño competente, evidenciando el logro de los elementos básicos del indicador, pero con omisiones, dificultades o errores.
Desempeño incipiente	**30%**	Presenta importantes omisiones, dificultades o errores en el desempeño, que no permiten evidenciar los elementos básicos del logro del indicador, por lo que no puede ser considerado competente.
Desempeño no logrado	**0%**	Presenta ausencia o incorrecto desempeño.

Indicador de Evaluación	Categorías de Respuesta					Ponderación Indicador de Evaluación
	Muy buen desempeño 100%	**Buen desempeño 80%**	**Desempeño aceptable 60%**	**Desempeño incipiente 30%**	**Desempeño no logrado 0%**	
Define en application.yml o clase @Configuration los nombres de colas, exchanges y bindings de forma centralizada.	Todos los nombres de colas, exchanges y bindings están definidos de manera centralizada en application.yml o en una clase de configuración única. No hay valores sueltos en el código. La organización es clara y consistente	La mayoría de los nombres está centralizada en el archivo de configuración o en la clase @Configuration. Existen pocos valores aislados en el código que no afectan de forma importante el mantenimiento.	Existe una mezcla evidente entre nombres centralizados y nombres escritos directamente en código. La configuración es usable pero dificulta el mantenimiento y la lectura.	La mayoría de los nombres de colas y exchanges está escrita directamente en las clases de negocio. El archivo de configuración se usa poco o nada.	No se utiliza configuración centralizada. Todos los nombres están dispersos en el código de forma manual.	12%

	para todos los microservicios.					
Declara beans de configuración de RabbitMQ con Queue, Exchange y Binding para cada caso de uso.	Existen clases de configuración que declaran beans de Queue, Exchange y Binding para cada caso de uso definido en el sistema. Cada ruta de mensajería está claramente identificada y documentada en código.	La mayoría de los casos de uso tiene sus beans de Queue, Exchange y Binding definidos. Faltan algunos escenarios poco utilizados o secundarios.	Solo los casos principales están configurados con beans explícitos. Otros usos dependen de configuraciones implícitas o incompletas.	Hay muy pocos beans de configuración. La mayor parte de la interacción con RabbitMQ se realiza desde el código de negocio sin una configuración clara.	No hay clases de configuración que declaren beans de Queue, Exchange o Binding.	13%
El código del microservicio no mezcla la lógica de negocio con la configuración de mensajería. Hay clases de configuración separadas.	La configuración de mensajería está completamente aislada en clases específicas. Los servicios de negocio solo consumen interfaces o componentes ya configurados. No se observa lógica de configuración en controladores o servicios de dominio.	En general la separación es clara. Existen pequeños fragmentos de configuración en clases de negocio que no afectan el diseño global.	La separación es parcial. Parte de la configuración está en clases dedicadas y otra parte se implementa directamente dentro de servicios o controladores.	Se observa una mezcla importante de lógica de negocio con instrucciones de configuración de colas y exchanges. La estructura dificulta el mantenimiento.	No existe separación. Toda la configuración de mensajería se encuentra incrustada en la lógica de negocio.	10%
Los consumidores están implementados con @RabbitListener u otra anotación equivalente y se agrupan por dominio funcional.	Todos los consumidores están correctamente implementados con @RabbitListener u otra anotación adecuada. Se organizan de manera clara por	La mayoría de los consumidores está implementada con @RabbitListener y se observa una organización por dominio funcional.	Los consumidores funcionan pero se encuentran mezclados dentro de un mismo paquete o clase sin una separación clara por dominios. La	Los consumidores están implementados de forma dispersa. No existe un criterio claro de organización por dominio y es difícil	No se utilizan @RabbitListener ni anotaciones equivalentes para implementar consumidores o directamente no se implementa ningún consumidor.	15%

	dominio funcional o contexto de negocio. La estructura del código facilita el mantenimiento y refleja una separación coherente de responsabilidades.	Existen pocos casos donde la agrupación podría ser más consistente.	estructura es comprensible pero no óptima.	identificar a qué parte del sistema pertenece cada consumidor.		
En los consumidores se maneja la confirmación de mensajes. Se ve uso de ACK y manejo explícito de errores.	El manejo de confirmación está completamente implementado. Cada consumidor gestiona ACK de forma explícita. Los errores se manejan correctamente con rutas diferenciadas y decisiones claras entre reintentos, NACK y envío a DLQ. La lógica de error está documentada y es consistente.	Los consumidores manejan ACK y controlan errores de manera adecuada en la mayoría de los casos. Existe alguna omisión menor en escenarios poco frecuentes que no afecta el funcionamiento general.	Los consumidores realizan ACK automáticos o parciales. El manejo de errores existe pero es limitado y no cubre todos los casos esperados. Puede haber requeue innecesario o falta de control cuando aparece un error.	El manejo de confirmación es muy básico. Los consumidores no distinguen entre errores recuperables o no recuperables. Se observan respuestas imprecisas que pueden provocar reprocesos o pérdida de mensajes.	No se implementa ACK ni manejo explícito de errores. Los consumidores procesan mensajes sin control de flujo y sin mecanismos para gestionar fallas.	20%
El microservicio administrador expone endpoints REST bien definidos por ejemplo POST /queues o DELETE /queues/{name}.	El microservicio administrador expone una API REST clara y coherente. Los endpoints tienen rutas, verbos HTTP y nombres de recursos bien definidos. La estructura sigue buenas prácticas de diseño REST y	Los endpoints están bien definidos en su mayoría. Hay detalles menores en nombres o verbos que no afectan el uso de la API.	La API es funcional pero presenta inconsistencias en nombres de rutas, verbos o estructura de los recursos. La intención se entiende pero no se alinea completamente con un diseño	Los endpoints son confusos o no siguen una estructura clara. Es difícil deducir qué hace cada ruta solo leyendo el código.	El microservicio administrador no expone endpoints REST para administrar colas o no implementa este microservicio.	13%

	está documentada.		REST cuidadoso.			
La lógica para crear colas y exchanges está encapsulada en un servicio por ejemplo RabbitAdminService.	Toda la lógica de creación, actualización y eliminación de colas y exchanges está encapsulada en un servicio dedicado. Los controladores llaman a métodos de alto nivel y no conocen detalles internos de RabbitMQ.	La mayoría de las operaciones de administración está concentrada en un servicio específico. Existen algunas llamadas directas a la librería de RabbitMQ desde otros puntos del código.	Parte de la lógica de administración está en un servicio y parte se reparte entre controladores o utilidades. La responsabilidad del servicio no es completamente clara.	Las operaciones de administración se implementan directamente en controladores o clases no especializadas. El servicio dedicado es inexistente o casi no se utiliza.	No existe un servicio dedicado para administrar colas y exchanges.	10%
La API valida los parámetros de entrada para evitar crear colas con nombres vacíos o configuraciones inválidas.	Los endpoints del microservicio administrador validan todos los parámetros de entrada. Se usan DTO con anotaciones de validación o lógica equivalente. No es posible crear colas con nombres vacíos ni con configuraciones inválidas. Los mensajes de error son claros.	La validación cubre los casos más importantes. Algunos parámetros secundarios no se validan pero no generan fallas críticas.	La API realiza validaciones básicas. Es posible que se cuelen configuraciones poco claras aunque la mayoría de los errores evidentes se detecta.	La validación es mínima. Es fácil crear colas con nombres inconsistentes o parámetros poco seguros.	No se valida la entrada. El backend acepta cualquier dato sin controles previos.	7%
					Total	**100%**